



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



METAHEURÍSTICAS

Actividad Autónoma 4

Nombre de la actividad: Implementación y evaluación de algoritmos de trayectoria en problemas benchmark

Unidad 2: Algoritmo de Trayectoria

Tema 2: Implementación y Aplicaciones



FACULTAD DE
Ingeniería

Nombre: Lenin Lopez

Fecha: 03/06/2026

Carrera: Ciencia de Datos e IA

Periodo académico: 2026-1S

Semestre: 4to.



Objetivo de la actividad:

Implementar y aplicar algoritmos de trayectoria en problemas de optimización estandarizados, evaluando su desempeño con métricas de precisión, eficiencia y robustez, y reflexionar críticamente sobre su adecuación en distintos contextos prácticos.

Recursos o temas que debe haber estudiado antes de hacer la actividad:

- Fundamentos de los algoritmos de trayectoria: Hill Climbing, Simulated Annealing (SA) y Tabu Search (TS).
- Conceptos de función de costo, criterios de parada y ajuste de parámetros.
- Conocimiento de problemas benchmark: TSP, Job Shop Scheduling, Bin Packing.
- Uso básico del lenguaje Python o R para programar o adaptar algoritmos de optimización.

Formato de entrega: PDF, Adjuntar archivo del código (.py o .R).

Instrucciones:

1. Selección del problema benchmark

- Elija uno de los siguientes problemas:
 - Traveling Salesman Problem (TSP) – versión reducida (5–10 nodos).

2. • Implementación de algoritmos

- Programe en Python o R al menos dos algoritmos de trayectoria (ej. SA y TS).

TSP	SA	tabu
<pre>import time import random import math from typing import Tuple, List</pre>	<pre>import math import random from typing import List, Tuple</pre>	<pre>import time import random from typing import List, Tuple from tsp import tour_length</pre>

```
from tsp import
tour_length, swap_neighbor

def simulated_annealing(D:
List[List[float]],
max_iters: int = 10000,
T0: float = 100.0, alpha:
float = 0.995) ->
Tuple[List[int], float,
int]:
    n = len(D)
    current =
list(range(n))
    random.shuffle(current
)
    current_cost =
tour_length(current, D)
    best = current.copy()
    best_cost =
current_cost
    T = T0
    it = 0
    start = time.time()
    while it < max_iters
and T > 1e-8:
        neighbor =
swap_neighbor(current)
        neighbor_cost =
tour_length(neighbor, D)
        delta =
neighbor_cost -
current_cost
        if delta < 0 or
random.random() <
math.exp(-delta / T):
            current =
neighbor
            current_cost =
neighbor_cost
            if
current_cost < best_cost:
                best =
current.copy()
                best_cost
= current_cost
                T *= alpha
                it += 1
                elapsed = time.time()
- start
```

```
def euclidean(a:
Tuple[float,float], b:
Tuple[float,float]) ->
float:
    return
math.hypot(a[0]-b[0],
a[1]-b[1])

def generate_points(n:
int, seed: int = None,
scale: int = 100) ->
List[Tuple[float,float]]
:
    if seed is not None:
        random.seed(seed
)
    return
[(random.random()*scale,
random.random()*scale)
for _ in range(n)]

def
distance_matrix(points:
List[Tuple[float,float]]
) -> List[List[float]]:
    n = len(points)
    D = [[0.0]*n for _
in range(n)]
    for i in range(n):
        for j in
range(n):
            if i != j:
                D[i][j]
= euclidean(points[i],
points[j])
    return D

def tour_length(tour:
List[int], D:
List[List[float]]) ->
float:
    n = len(tour)
    s = 0.0
    for i in range(n):
        a = tour[i]
        b =
tour[(i+1)%n]
        s += D[a][b]
    return s
```

```
def swap_move(tour: List[int], i: int, j:
int) -> List[int]:
    new = tour.copy()
    new[i], new[j] = new[j], new[i]
    return new

def tabu_search(D: List[List[float]],
max_iters: int = 1000, tabu_tenure: int =
7) -> Tuple[List[int], float, int]:
    n = len(D)
    current = list(range(n))
    random.shuffle(current)
    best = current.copy()
    best_cost = tour_length(best, D)
    tabu_list = {}
    it = 0
    start = time.time()
    while it < max_iters:
        neighborhood = []
        for i in range(n-1):
            for j in range(i+1, n):
                neighbor =
swap_move(current, i, j)
                cost =
tour_length(neighbor, D)
                neighborhood.append((cost
, i, j, neighbor))
                neighborhood.sort(key=lambda x:
x[0])
                moved = False
                for cost, i, j, neighbor in
neighborhood:
                    move = (i, j)
                    if move not in tabu_list or
cost < best_cost:
                        current = neighbor
                        if cost < best_cost:
                            best =
neighbor.copy()
                            best_cost = cost
                            tabu_list[move] = it +
tabu_tenure
                            moved = True
                            break
                    # decrement / remove expired
                    expired = [m for m, expiry in
tabu_list.items() if expiry <= it]
                    for m in expired:
                        del tabu_list[m]
```



```
return best,  
best_cost, it, elapsed  
  
if __name__ == '__main__':  
    print('Run sa module  
as library')
```

```
def random_tour(n: int)  
-> List[int]:  
    tour =  
list(range(n))  
    random.shuffle(tour)  
    return tour  
  
def swap_neighbor(tour:  
List[int]) -> List[int]:  
    n = len(tour)  
    i =  
random.randrange(n)  
    j =  
random.randrange(n)  
    while j == i:  
        j =  
random.randrange(n)  
    new = tour.copy()  
    new[i], new[j] =  
new[j], new[i]  
    return new  
  
def  
two_opt_neighbor(tour:  
List[int]) -> List[int]:  
    n = len(tour)  
    i =  
random.randrange(0, n-1)  
    j =  
random.randrange(i+1, n)  
    new = tour.copy()  
    new[i:j+1] =  
reversed(new[i:j+1])  
    return new
```

```
if not moved:  
    break  
    it += 1  
elapsed = time.time() - start  
return best, best_cost, it, elapsed  
  
if __name__ == '__main__':  
    print('Run tabu module as library')
```

- Documente el diseño de la función de costo, el vecindario y el criterio de parada.

3. • Ejecución y resultados

- Ejecute cada algoritmo al menos 5 veces sobre el mismo problema.
- Registre:
 - Calidad de la solución obtenida (valor de la función de costo).
 - Tiempo de ejecución.
 - Número de iteraciones.

Algoritmo	Runs	Mejor costo	Costo medio	Tiempo medio (s)	Iteraciones media
Simulated Annealing	5	253.2775	253.2775	0.0219	4594.0
Tabu Search	5	253.2775	253.2775	0.1946	2000.0

4. • Análisis comparativo

- Presente los resultados en una tabla comparativa.
- Analice las diferencias en precisión, eficiencia y robustez.

Algoritmo	Precisión (calidad)	Eficiencia (tiempo/recursos)	Robustez (consistencia/ajuste de parámetros)
Simulated Annealing (SA)	Alta — buena capacidad para escapar de mínimos locales	Alta — bajo coste por iteración, requiere más iteraciones	Media — sensible a temperatura y programación de enfriamiento
Tabu Search (TS)	Alta — intensificación local efectiva	Media-Baja — costo por iteración mayor si se examina todo el vecindario	Alta — estable si se ajusta la tenencia tabú y el vecindario

5. • Reflexión crítica

- Redacte un ensayo (mínimo de 300 palabras) sobre la aplicabilidad de los algoritmos de trayectoria en problemas reales de logística, robótica o planificación de recursos.

Los algoritmos de trayectoria y las metaheurísticas aplicadas al TSP tienen una relevancia directa en problemas reales de logística, robótica y planificación de recursos, porque ofrecen un equilibrio práctico entre calidad de solución y coste computacional. En logística, por ejemplo, rutas de reparto y recolección requieren soluciones cercanas al óptimo en tiempos limitados: aquí, una combinación de heurística inicial (Nearest Neighbor) seguida de una mejora local (2-opt/3-opt) puede proporcionar rutas útiles en segundos, mientras que métodos más costosos (GA, ACO, TS) pueden reservarse para windows de optimización nocturna o planificación estratégica donde el tiempo no es tan crítico.

En robótica la planificación de trayectorias tiene además restricciones geométricas y dinámicas (obstáculos, cinemática del robot) que convierten el problema en variantes del TSP o en problemas de ruteo con restricciones. Las metaheurísticas son valiosas porque permiten incorporar restricciones adicionales en la evaluación de soluciones y explorar espacios de alta dimensionalidad donde métodos exactos no son prácticos. Sin embargo, en entornos con fuerte requerimiento en tiempo real, es común usar políticas híbridas: planificador rápido para emergencia y optimizador off-line para refinar trayectorias.

En planificación de recursos (asignación de tareas, secuenciación de máquinas), la robustez y la consistencia son tan importantes como la calidad puntual de la solución; aquí, TS puede ser preferible por su capacidad para intensificar y diversificar controladamente, mientras que SA o GA aportan flexibilidad para escapar de mínimos locales en espacios complejos.

Finalmente, la aplicabilidad práctica exige más que un buen algoritmo: requiere modelado realista, evaluaciones delta para eficiencia, validación con datos reales y pipelines automatizados para ajuste de parámetros. Las decisiones entre velocidad y calidad, la tolerancia a subóptimos y la necesidad de explicabilidad condicionan la elección del método en cada dominio. En resumen, las metaheurísticas aportan herramientas potentes y flexibles, pero su éxito en la práctica depende tanto de su implementación eficiente como de una integración cuidadosa con las restricciones y procesos operativos del problema a resolver.



Bibliografía:

Guilmeau, T., Chouzenoux, E., & Elvira, V. (2022). Proximal-based adaptive simulated annealing for global optimization. *ICASSP 2022 – IEEE International Conference on Acoustics, Speech and Signal Processing*, 5453–5457. <https://doi.org/10.1109/ICASSP43922.2022.9746626>

Venkateswarlu, C. (2021). A metaheuristic tabu search optimization algorithm: Applications to chemical and environmental processes. En *Engineering Problems—Uncertainties, Constraints and Optimization Techniques*. IntechOpen. <https://doi.org/10.5772/intechopen.98240>

Xu, L., Wang, Z., Chen, X., & Lin, Z. (2022). Multi-parking lot and shelter heterogeneous vehicle routing problem with split pickup under emergencies. *IEEE Access*, 10, 36073–36090. <https://doi.org/10.1109/ACCESS.2022.3163715>

Rowold, M., Ögretmen, L., Kerbl, T., & Lohmann, B. (2022). Efficient spatiotemporal graph search for local trajectory planning on oval race tracks. *Actuators*, 11(11), 319. <https://doi.org/10.3390/act11110319>

Hu, T., Ochoa, G., & Banzhaf, W. (2023). Phenotype search trajectory networks for linear genetic programming. *arXiv*. <https://doi.org/10.48550/arXiv.2211.08516>

Rúbrica de evaluación

Componente de aprendizaje:	Autónomo	X	Contacto con el Docente	
Nombre de la Unidad:	Unidad 2: Algoritmos de Trayectoria.			
Resultado(s) de aprendizaje:	- Distinguir las características esenciales de los algoritmos de trayectoria, diferenciándolos de otros enfoques metaheurísticos poblacionales. - Aplicar algoritmos de trayectoria a problemas concretos, evaluando su eficiencia y adecuación a distintos contextos de optimización.			
Nombre de la Actividad:	Implementación y evaluación de algoritmos de trayectoria en problemas benchmark.			



Criterios de Evaluación	Escala de Valoración						
	Excelente (10 - 9,1)	Bueno (9 - 8,1)	Satisfactorio (8 - 7)	Necesita mejorar (6,9 - 0,1)	No entrega (0)	Puntaje	Comentarios (SIGEA)
1. Resumen conceptual	Explica claramente fundamentos, motivaciones y bases en biología, física y ecología con redacción académica impecable.	Presenta fundamentos y motivaciones con algunos vacíos o redacción mejorable.	Explica de forma parcial, omite aspectos clave o tiene errores menores.	Explicación incompleta, con errores conceptuales o poco rigor académico.	No entrega		
2. Tabla comparativa	Incluye al menos 5 diferencias bien argumentadas entre bioinspirados y otras metaheurísticas, con ejemplos claros.	Presenta 4 o 5 diferencias, pero algunas poco desarrolladas.	Presenta menos de 4 diferencias o con errores notables.	Comparación muy incompleta o incorrecta.	No entrega		
3. Aplicaciones recientes	Selecciona 2 algoritmos y describe aplicaciones actuales (≤5 años) con detalle y ventajas identificadas.	Selecciona 2 algoritmos con aplicaciones, pero con explicaciones limitadas.	Selecciona solo 1 algoritmo o aplicaciones poco actualizadas.	Aplicaciones vagas, sin evidencia o sin relación con el tema.	No entrega		
4. Reflexión crítica (Ejes: Innovación / Impacto social)	Reflexión profunda, conecta teoría, práctica e impacto social y tecnológico.	Reflexión adecuada, con aportes prácticos pero menos profundos.	Reflexión general, poco crítica.	Reflexión superficial, sin análisis.	No entrega		
5. Bibliografía (Eje: Ética y valores / Comunicación académica)	Incluye ≥5 fuentes académicas en APA 7, actualizadas y correctamente citadas.	Incluye 4-5 fuentes, con pequeños errores de formato.	Incluye 3 fuentes, con errores de citación.	Incluye menos de 3 fuentes o sin formato APA.	No entrega		

Puntaje total

Los criterios 4 y 5 están alineados a los ejes de formación del Modelo Educativo UNACH "Introspección y Prospectiva" y responden principalmente a dos de los siguientes ejes:

1. Ambiente;
2. Autonomía y adaptabilidad;
3. Comunicación;
4. Desarrollo humano;
5. Ética y valores;
6. Emprendimiento;
7. Inter y multidisciplinariedad;
8. Innovación;
9. Inclusión e interculturalidad;
10. Investigación;
11. Impacto social; 12. Tecnologías.